République du Bénin

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Nationale des Sciences, Technologies,
Ingénierie et Mathématiques d'Abomey (UNSTIM)

École Nationale des Génies Mathématiques et Modélisation (ENSGMM)

Implémentation du modèle d'infection par le VIH-SIDA

Modèles intra-hôte et de population : analyse mathématique, stabilité et résolution numérique avec BLAS, LAPACK et Intel MKL en Python

Équipe de projet – Groupe 5

ADOGOUN Déo Gracias Mahugnon
BIAHOU Ahossou J. Marie-Clémenceau
SYLLA IMOROU Izou Dine

Promotion : ENSGMM1 • **UE :** Outils de calcul scientifique
Année académique : 2026–2027
Sous la direction de : Dr. Carlos AGOSSOU

Table des matières

Introduction **1**

1 BLAS : Les Briques de Base de l'Algèbre Linéaire **2**
1.1 Qu'est-ce que BLAS ? . . . 2
1.2 Les trois niveaux de BLAS et l'intensité arithmétique . . . 2
1.3 Conventions de nommage de BLAS . . . 3

2 LAPACK : Le Solveur Matriciel Avancé **4**
2.1 Qu'est-ce que LAPACK ? . . . 4
2.2 Définitions des concepts mathématiques clés . . . 4
2.3 Les principales routines de LAPACK . . . 4
2.4 La pile logicielle : du problème au matériel . . . 5

3 Intel oneMKL : L'Implémentation Optimisée Industrielle **6**
3.1 Pourquoi Intel MKL vient après BLAS et LAPACK ? . . . 6
3.2 Les secrets de la performance d'Intel MKL . . . 6
3.3 Comparaison avec les alternatives du marché . . . 6

4 Installation et Utilisation en Python **8**
4.1 Comment vérifier la bibliothèque active sous Python . . . 8
4.2 Appel direct des routines LAPACK depuis Python . . . 8

5 Contexte Biologique du VIH-SIDA **10**
5.1 Comment le virus attaque-t-il l'organisme ? . . . 10
5.2 Les trois phases cliniques chez un patient non traité . . . 10
5.3 Pourquoi modéliser à deux échelles ? . . . 10

6 Le Modèle Intra-Hôte : Formulation et Analyse Mathématique **11**
6.1 Les équations différentielles du modèle . . . 11
6.2 Calcul détaillé de $\mathcal{R}_0$ par la Matrice de Prochaine Génération (NGM) . . . 11
6.3 Stabilité et équilibres stationnaires . . . 13

7 Méthodes Numériques et Solveurs LAPACK **15**
7.1 Résolution de systèmes linéaires avec DGESV . . . 15
7.2 Équilibrage de matrice avec DGEEQU . . . 15
7.3 Méthode de Newton avec DGESV et calcul spectral DGEEV . . . 16

8 Intégration Numérique des Équations et Raideur **17**
8.1 Qu'est-ce que la raideur numérique ? . . . 17
8.2 Pourquoi les schémas explicites explosent-ils ? . . . 17
8.3 Le remède : Euler implicite résolu avec DGESV . . . 17

9 Résultats et Interprétations Biomédicales **19**
9.1 Dynamique de primo-infection et portrait de phase 19
9.2 Réponse au traitement et danger de l'interruption 19

10 Modèle Épidémiologique de Population : S, I_1, I_2 **22**
10.1 Formulation conforme au cours 22
10.2 Nombre de reproduction de base et simulations sur 50 ans 23

11 Benchmark Haute Performance : Gains de Vitesse Réels **24**
11.1 Protocole de mesure expérimental 24
11.2 Tableau comparatif des accélérations (Speedup) 24
11.3 Passage à l'échelle multi-cœurs (Scalabilité OpenMP) 24

Conclusion et Perspectives **26**

Références Bibliographiques **27**

A Extraits Commentés du Code Source Principal **28**
A.1 Modèle intra-hôte et Euler Implicite (`src/intra_host.py`) 28
A.2 Benchmark de DGEMM et DGESV (`src/benchmark.py`) 28

B Guide Pratique d'Exécution et Reproductibilité **29**

Introduction

Résumé en clair : de quoi parle ce projet ?

Ce travail étudie la dynamique du virus du SIDA (le VIH) à deux échelles complémentaires :

1. **À l'échelle du patient (modèle intra-hôte) :** comment le virus attaque les globules blancs (CD4+) dans le sang et comment les traitements antirétroviraux bloquent cette infection.
2. **À l'échelle de la population (modèle épidémiologique) :** comment l'épidémie se propage entre personnes saines, infectées sans symptômes et malades du SIDA sur plusieurs décennies.

Pour simuler ces équations de manière exacte et rapide, nous utilisons les bibliothèques professionnelles de calcul scientifique : **BLAS**, **LAPACK** et **Intel MKL**.

Pourquoi ce document est-il rédigé ainsi ? Ce rapport a été pensé pour être lu et compris par tout étudiant en première année d'ingénierie mathématique (ENSGMM1), sans prérequis lourd en informatique haute performance. Chaque notion technique est expliquée avec des mots simples dès sa première apparition.

Les 4 objectifs clés du travail :

1. Comprendre le rôle et l'articulation de la pile logicielle standard : **BLAS** (niveau de base), **LAPACK** (solveurs matriciels) et **Intel MKL** (moteur de calcul ultra-rapide).
2. Écrire et résoudre les équations différentielles biologiques en évitant les instabilités et les explosions de calcul (*raideur numérique*).
3. Calculer à la main et par ordinateur le seuil critique $\mathcal{R}_0$ et l'efficacité minimale de traitement $\eta_c = 75\,\%$ nécessaire pour guérir un patient.
4. Mesurer les gains de vitesse réels : montrer qu'un code basé sur BLAS/LAPACK s'exécute jusqu'à plus de **1 000 fois plus vite** qu'un code Python ordinaire.

Chapitre 1

BLAS : Les Briques de Base de l'Algèbre Linéaire

1.1 Qu'est-ce que BLAS ?

Définition simple de BLAS

BLAS signifie *Basic Linear Algebra Subprograms* (Sous-programmes élémentaires d'algèbre linéaire). Créée dès 1979, c'est une liste standardisée d'opérations mathématiques de base sur les vecteurs et les matrices (additions, produits scalaires, multiplications de matrices).

Au lieu que chaque scientifique réécrive sa propre boucle `for` pour multiplier deux matrices, BLAS définit un standard universel. Ainsi, un programme écrit en Python ou en C peut appeler la fonction standard `DGEMM`, et l'ordinateur utilisera la version la plus optimisée pour son processeur.

1.2 Les trois niveaux de BLAS et l'intensité arithmétique

BLAS est structuré en trois niveaux, selon la dimension des objets manipulés (tableau 1.1).

Table 1.1 – Les trois niveaux de BLAS et leur intensité arithmétique.

Niveau	Opérations	Données	Calculs	Intensité	Exemple typique
1	Vecteur – Vecteur	$O(n)$	$O(n)$	$O(1)$ (Faible)	`DAXPY` : $y \leftarrow \alpha x + y$
2	Matrice – Vecteur	$O(n^2)$	$O(n^2)$	$O(1)$ (Faible)	`DGEMV` : $y \leftarrow \alpha Ax + \beta y$
3	Matrice – Matrice	$O(n^2)$	$O(n^3)$	$\mathbf{O(n)}$ (Élevée)	`DGEMM` : $C \leftarrow \alpha AB + \beta C$

Définition simple : Intensité arithmétique et Cache

- **La mémoire cache :** C'est une mémoire miniature située directement au cœur du processeur (niveaux L1, L2, L3). Elle est des dizaines de fois plus rapide que la mémoire vive (RAM), mais sa taille est très réduite (quelques kilo-octets ou méga-octets).
- **L'intensité arithmétique :** C'est le nombre d'opérations de calcul (additions, multiplications) effectuées pour chaque octet de donnée transféré depuis la RAM.

Pourquoi le Niveau 3 (DGEMM) est-il infiniment plus efficace ?

- Aux **niveaux 1 et 2**, pour chaque nombre lu en mémoire RAM, le processeur ne fait qu'une seule opération. La mémoire RAM est trop lente par rapport au processeur : le processeur attend les données. On dit que le programme est **limité par la mémoire** (*Memory-Bound*).

— Au **niveau 3**, pour multiplier deux matrices $n \times n$, il y a $2n^2$ données en entrée, mais $2n^3$ opérations à faire. Le rapport calculs/données est d'ordre $O(n)$. Grâce à une technique appelée **découpage par blocs** (*cache blocking*), les sous-matrices sont chargées dans le cache L1 et réutilisées sur place plusieurs dizaines de fois sans retourner en RAM. Le processeur tourne alors à sa vitesse maximale. On dit qu'il est **limité par le calcul** (*Compute-Bound*).

1.3 Conventions de nommage de BLAS

Le nom de chaque routine indique précisément ce qu'elle fait :

`[Précision][Type de matrice][Opération]`

— **Précision (1ère lettre) :** `S` pour réel simple précision (32 bits), `D` pour réel double précision (64 bits, standard en science), `C` pour complexe simple, `Z` pour complexe double.
— **Type de matrice (2 lettres) :** `GE` (matrice générale quelconque), `SY` (symétrique), `TR` (triangulaire).
— **Opération :** `MM` (multiplication matrice-matrice), `MV` (matrice-vecteur).

Exemple : `DGEMM` = **D**ouble précision, matrice **GE**nérale, produit **M**atrice-**M**atrice.

Mini-résumé du Chapitre 1 : BLAS

1. BLAS est le standard universel des opérations élémentaires d'algèbre linéaire.
2. Le niveau 1 concerne les vecteurs, le niveau 2 les matrices-vecteurs, et le niveau 3 les matrices-matrices.
3. Seul le niveau 3 (ex. `DGEMM`) réutilise les données dans la mémoire cache du processeur, offrant des performances maximales.
4. Les noms des fonctions suivent une convention claire (ex : `DGEMM` pour le produit matriciel 64 bits).

Chapitre 2

LAPACK : Le Solveur Matriciel Avancé

2.1 Qu'est-ce que LAPACK ?

Définition simple de LAPACK

LAPACK signifie *Linear Algebra PACKage*. Alors que BLAS ne fait que des calculs simples (comme multiplier deux matrices), LAPACK résout des **vrais problèmes mathématiques** : résoudre un système linéaire $Ax = b$, trouver des valeurs propres, ou inverser une matrice.

Créée au début des années 1990 pour moderniser les anciens codes LINPACK et EISPACK, LAPACK est entièrement conçue pour s'appuyer sur le niveau 3 de BLAS. Au lieu de factoriser une matrice ligne par ligne (ce qui gaspillerait le temps d'accès mémoire), LAPACK découpe la matrice en blocs rectangulaires et délègue toutes les grosses opérations à `DGEMM`.

2.2 Définitions des concepts mathématiques clés

Les notions à connaître

- **Factorisation LU :** Décomposer une matrice carrée A sous la forme $PA = LU$, où P est une matrice de permutation de lignes (pivotage partiel), L est triangulaire inférieure avec des 1 sur la diagonale, et U est triangulaire supérieure. Une fois L et U connues, résoudre $Ax = b$ devient immédiat : on résout $Ly = Pb$ (descente), puis $Ux = y$ (remontée).
- **Conditionnement $\kappa(A)$:** Nombre ≥ 1 qui mesure la sensibilité de la solution face à une petite perturbation sur les données. Si $\kappa(A) \approx 1$, la matrice est très stable. Si $\kappa(A) \geq 10^8$, la matrice est dite **mal conditionnée** et les calculs numériques risquent de perdre toute précision.
- **Valeurs propres λ :** Nombres scalaires vérifiant $Av = \lambda v$ pour un vecteur propre non nul v. Leurs parties réelles renseignent directement sur la stabilité d'un système dynamique.

2.3 Les principales routines de LAPACK

Le tableau 2.1 résume les fonctions les plus importantes de LAPACK utilisées dans ce projet.

TABLE 2.1 – Principales fonctionnalités de LAPACK exploitées dans le projet.

Problème à résoudre	**Routine**	**Méthode mathématique**	**Complexité**
Résoudre $Ax = b$	`DGESV`	Factorisation LU avec pivotage partiel ($PA = LU$)	$\frac{2}{3}n^3$
Équilibrer une matrice	`DGEEQU`	Mise à l'échelle des lignes et colonnes	$O(n^2)$
Estimer le conditionnement	`DGECON`	Estimation rapide de $\|A^{-1}\|_1$ sans inverser	$O(n^2)$
Valeurs propres générales	`DGEEV`	Réduction de Hessenberg et algorithme QR décalé	$O(n^3)$
Décomposition SVD	`DGESDD`	Valeurs singulières par approche Diviser pour Régner	$O(n^3)$

2.4 La pile logicielle : du problème au matériel

La figure 2.1 montre comment les différents étages s'articulent logiquement.

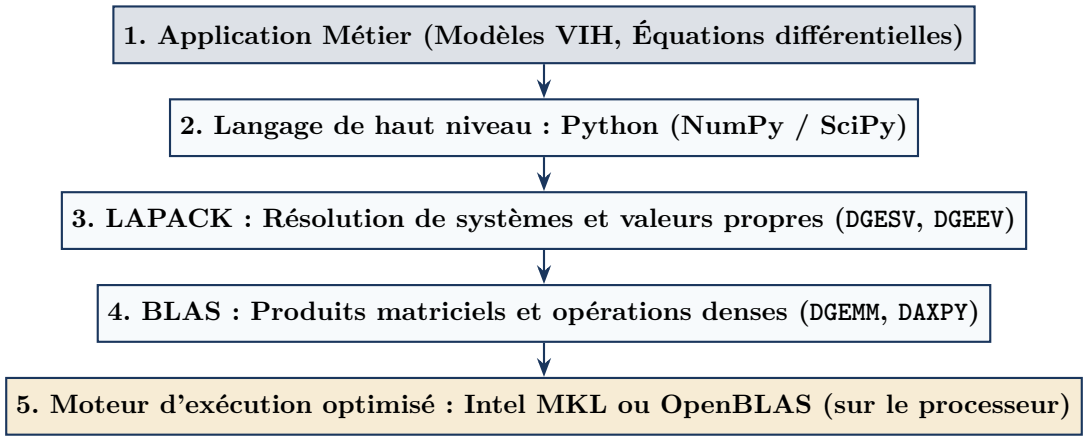


FIGURE 2.1 – Architecture en couches : de la modélisation biologique jusqu'aux circuits du processeur.

Mini-résumé du Chapitre 2 : LAPACK

1. LAPACK résout les vrais problèmes d'algèbre linéaire (systèmes $Ax = b$, valeurs propres, décompositions).
2. Elle s'appuie directement sur BLAS 3 (`DGEMM`) pour maximiser la vitesse de calcul.
3. La routine pilote `DGESV` résout les systèmes linéaires généraux par factorisation LU $PA = LU$.
4. `DGEEQU` permet d'équilibrer une matrice avant calcul pour éviter les erreurs d'arrondi.

Chapitre 3

Intel oneMKL : L'Implémentation Optimisée Industrielle

3.1 Pourquoi Intel MKL vient après BLAS et LAPACK ?

Comme expliqué dans les chapitres précédents :
— **BLAS** est une spécification (la liste des fonctions à avoir).
— **LAPACK** est une bibliothèque algorithmique de haut niveau.
— **Intel MKL (oneMKL)** est **l'implémentation industrielle ultra-optimisée** de BLAS et de LAPACK, développée par les ingénieurs d'Intel pour tirer 100 % des capacités de leurs processeurs.

3.2 Les secrets de la performance d'Intel MKL

Intel MKL accélère les calculs grâce à trois technologies matérielles :

1. **La vectorisation SIMD (AVX2, AVX-512) :** Au lieu de calculer un nombre à la fois, le processeur exécute une seule instruction qui traite 4 ou 8 nombres flottants simultanément dans de larges registres matériels (512 bits).
2. **Le parallélisme multi-cœurs (OpenMP) :** MKL découpe automatiquement les grandes matrices et distribue les blocs sur tous les cœurs du processeur (4, 8, 16 cœurs ou plus).
3. **Le répartiteur dynamique (*CPU Dispatcher*) :** Au lancement du programme, MKL détecte exactement le modèle de processeur présent et choisit automatiquement la version assembleur la plus rapide.

3.3 Comparaison avec les alternatives du marché

Le tableau 3.1 compare les principales implémentations disponibles.

Mini-résumé du Chapitre 3 : Intel MKL

1. Intel MKL est une implémentation propriétaire gratuite, optimisée pour processeurs x86-64.
2. Elle fournit BLAS et LAPACK avec vectorisation matérielle (AVX-512) et multi-threading OpenMP.
3. Elle détecte automatiquement le processeur pour exécuter les instructions assembleur les plus rapides.
4. Elle est la référence mondiale de performance pour le calcul scientifique sur CPU.

TABLE 3.1 – Comparaison des bibliothèques d'algèbre linéaire.

Bibliothèque	**Statut / Licence**	**Avantages et particularités**
Netlib (Référence)	Open-source	Code de référence en Fortran, sans vectorisation. Sert d'étalon de comparaison.
OpenBLAS	Open-source (BSD)	Très rapide, multiplateforme. Incluse par défaut lors de l'installation de NumPy via `pip`.
BLIS	Open-source (BSD)	Développée par l'Université du Texas, axée sur la modularité des micro-noyaux.
Intel oneMKL	Gratuit (Intel)	La plus performante sur architectures x86-64, optimisée en assembleur pour AVX-512 et multi-threading.

Chapitre 4

Installation et Utilisation en Python

4.1 Comment vérifier la bibliothèque active sous Python

Dans le monde Python, NumPy et SciPy sont pré-compilés avec une bibliothèque sous-jacente (généralement OpenBLAS ou Intel MKL). On peut vérifier la bibliothèque en 3 lignes de code :

```python
import numpy as np
from threadpoolctl import threadpool_info

# Affiche la configuration BLAS de NumPy
print(np.show_config())

# Affiche le moteur exact, la version et le nombre de threads
print(threadpool_info())
```

Explication ligne par ligne :

- `import numpy as np` : charge la bibliothèque de calcul numérique matriciel.
- `threadpool_info()` : interroge la mémoire vive pour savoir quelle bibliothèque partagée (OpenBLAS ou MKL) est réellement en train de tourner et combien de cœurs sont configurés.

4.2 Appel direct des routines LAPACK depuis Python

Plutôt que d'utiliser la fonction générique `np.linalg.solve`, SciPy permet d'appeler directement les routines Fortran originales de LAPACK, sans copie inutile en mémoire :

```python
from scipy.linalg import lapack
import numpy as np

# Matrice A en ordre Fortran (colonne par colonne) et vecteur second membre b
A = np.asfortranarray([[3.0, 1.0], [1.0, 2.0]], dtype=np.float64)
b = np.asfortranarray([9.0, 8.0], dtype=np.float64)

# 1. Résolution directe Ax = b avec la routine LAPACK DGESV
lu, piv, x, info = lapack.dgesv(A, b)
assert info == 0, "Échec : matrice singulière"
print("Solution x =", x) # Donne [2.0, 3.0]

# 2. Calcul des valeurs propres d'une matrice avec DGEEV
wr, wi, vl, vr, info = lapack.dgeev(A)
valeurs_propres = wr + 1j * wi
print("Valeurs propres =", valeurs_propres)
```

Explication pas à pas du code :

- `np.asfortranarray(...)` : convertit la matrice en stockage par colonnes (standard historique Fortran), ce qui évite toute recopie de données lors du passage à LAPACK.

— `lapack.dgesv(A, b)` : exécute la factorisation LU $PA = LU$ et résout le système. Elle renvoie la solution `x`, les indices de permutation `piv`, et un code d'erreur `info` (égal à 0 si tout s'est bien passé).
— `lapack.dgeev(A)` : calcule les valeurs propres décomposées en partie réelle (`wr`) et imaginaire (`wi`).

Mini-résumé du Chapitre 4 : Utilisation en Python

1. Python se lie automatiquement à BLAS et LAPACK via NumPy et SciPy.
2. Le module `scipy.linalg.lapack` permet d'appeler directement les fonctions Fortran d'origine.
3. L'utilisation de `order='F'` (Fortran) garantit zéro surcoût de mémoire et une vitesse maximale.

Chapitre 5

Contexte Biologique du VIH-SIDA

5.1 Comment le virus attaque-t-il l'organisme ?

Le VIH (Virus de l'Immunodéficience Humaine) est un rétrovirus qui cible principalement les **lymphocytes T CD4+**, des globules blancs indispensables pour alerter et coordonner l'ensemble du système immunitaire.

— Un virion libre pénètre dans une cellule saine T.
— Il injecte son ARN, le transforme en ADN et force la cellule à produire des centaines de nouveaux virions.
— La cellule infectée finit par mourir rapidement, libérant une nouvelle vague de virus.

5.2 Les trois phases cliniques chez un patient non traité

1. **Phase aiguë (semaines 2 à 6) :** Multiplication explosive du virus. La charge virale atteint un pic énorme, tandis que les lymphocytes CD4+ chutent brutalement.
2. **Phase chronique (années 2 à 10) :** Le système immunitaire réagit et contient le virus à un niveau stationnaire appelé point de consigne (*set-point*). Le patient ne ressent aucun symptôme visible, mais le virus continue de détruire lentement les CD4+.
3. **Stade SIDA :** Lorsque le taux de CD4+ passe sous le seuil fatal de 200 cellules/μL, le système immunitaire s'effondre. Des maladies opportunistes (tuberculose, pneumonies) apparaissent, conduisant au décès sans traitement.

5.3 Pourquoi modéliser à deux échelles ?

— **À l'échelle intra-hôte (dans le corps du patient) :** Les interactions ont lieu en jours ou en semaines. Le but est d'évaluer si un médicament antirétroviral parvient à stopper la prolifération virale.
— **À l'échelle de la population (épidémiologique) :** La transmission sexuelle entre individus opère sur des décennies. Le but est de prédire l'impact de politiques sanitaires (dépistage, préservatifs, traitements).

Mini-résumé du Chapitre 5 : Contexte Biologique

1. Le VIH détruit les lymphocytes T CD4+, protecteurs de notre système immunitaire.
2. Sans traitement, la maladie évolue inévitablement vers le stade SIDA mortel.
3. Les trithérapies modernes (antirétroviraux) bloquent l'infection de nouvelles cellules.
4. Deux échelles de temps coexistent : l'échelle du patient (jours) et l'échelle de la société (années).

Chapitre 6

Le Modèle Intra-Hôte : Formulation et Analyse Mathématique

6.1 Les équations différentielles du modèle

Le modèle de référence de Nowak, May et Perelson comporte trois variables :
— $T(t)$: concentration de lymphocytes T CD4+ sains (cellules/μL).
— $I(t)$: concentration de cellules infectées productrices de virus (cellules/μL).
— $V(t)$: charge virale libre dans le plasma sanguin (virions/μL).
Le système s'écrit sous la forme de trois équations différentielles non-linéaires couplées :

$$\begin{cases} \dfrac{dT}{dt} = \underbrace{s}_{\text{source}} - \underbrace{dT}_{\text{mort naturelle}} - \underbrace{(1-\eta)\beta TV}_{\text{nouvelles infections}} \\ \dfrac{dI}{dt} = \underbrace{(1-\eta)\beta TV}_{\text{nouvelles infections}} - \underbrace{\delta I}_{\text{mort des cellules infectées}} \\ \dfrac{dV}{dt} = \underbrace{pI}_{\text{production virale}} - \underbrace{cV}_{\text{élimination du virus}} \end{cases} \tag{6.1}$$

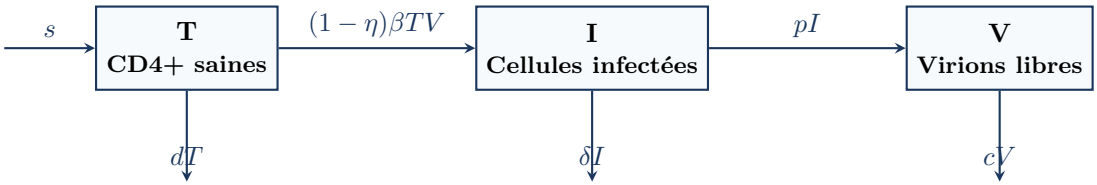


FIGURE 6.1 – Schéma de transfert compartimental du modèle intra-hôte.

6.2 Calcul détaillé de $\mathcal{R}_0$ par la Matrice de Prochaine Génération (NGM)

Définition de $\mathcal{R}_0$

Le **nombre de reproduction de base** $\mathcal{R}_0$ est le nombre moyen de cellules infectées secondaires créées par une seule cellule infectée introduite dans un tissu totalement sain.
— Si $\mathcal{R}_0 < 1$: Chaque cellule infectée engendre moins d'une nouvelle cellule infectée. L'infection s'éteint d'elle-même.
— Si $\mathcal{R}_0 > 1$: L'infection prolifère et s'installe durablement.

TABLE 6.1 – Paramètres biomédicaux du modèle intra-hôte.

Paramètre	Signification biologique	Valeur	Unité	Interprétation courante
s	Production de CD4+ saines	10	cellules/μL/jour	La moelle osseuse produit 10 CD4+/μL/j
d	Mortalité naturelle des CD4+	0,01	jour^{-1}	Durée de vie moyenne d'un CD4+ = $1/d = 100$ j
β	Infectivité virale	$4{,}6 \times 10^{-5}$	μL/virion/jour	Probabilité de rencontre et pénétration
δ	Mortalité des cellules infectées	0,50	jour^{-1}	Cellule infectée détruite en 2 jours ($1/\delta$)
p	Production de virions par cellule	1000	virions/cellule/jour	Une cellule infectée libère 1000 virions/j
c	Clairance virale plasmatique	23	jour^{-1}	Demi-vie virale très courte : 43 minutes
η	Efficacité du traitement	0 à 1	sans unité	$\eta = 0$ (sans ARV), $\eta = 0{,}95$ (trithérapie à 95%)

Détail pas à pas des 7 étapes de la méthode NGM (Diekmann et al.) :

Étape 1 : Isoler les compartiments infectés. Les variables infectées sont $y = (I, V)^T$. L'état sain sans infection (noté DFE) est $E_0 = (T_0, 0, 0)$ avec $T_0 = s/d = 10/0{,}01 = 1\,000$ cellules/μL.

Étape 2 : Séparer les nouvelles infections ($\mathcal{F}$) et les transferts/morts ($\mathcal{V}$). On réécrit les équations sous la forme $\frac{d}{dt}\begin{pmatrix} I \\ V \end{pmatrix} = \mathcal{F}(y) - \mathcal{V}(y)$:

$$\mathcal{F} = \begin{pmatrix} (1-\eta)\beta T V \\ 0 \end{pmatrix}, \quad \mathcal{V} = \begin{pmatrix} \delta I \\ -pI + cV \end{pmatrix}$$

Étape 3 : Calculer les matrices Jacobiennes F et V_m au point sain E_0. En dérivant par rapport à I et V, puis en posant $T = T_0$:

$$F = \begin{pmatrix} \frac{\partial \mathcal{F}_1}{\partial I} & \frac{\partial \mathcal{F}_1}{\partial V} \\ \frac{\partial \mathcal{F}_2}{\partial I} & \frac{\partial \mathcal{F}_2}{\partial V} \end{pmatrix}_{E_0} = \begin{pmatrix} 0 & (1-\eta)\beta T_0 \\ 0 & 0 \end{pmatrix}$$

$$V_m = \begin{pmatrix} \frac{\partial \mathcal{V}_1}{\partial I} & \frac{\partial \mathcal{V}_1}{\partial V} \\ \frac{\partial \mathcal{V}_2}{\partial I} & \frac{\partial \mathcal{V}_2}{\partial V} \end{pmatrix}_{E_0} = \begin{pmatrix} \delta & 0 \\ -p & c \end{pmatrix}$$

Étape 4 : Inverser explicitement la matrice V_m. Comme V_m est triangulaire inférieure 2×2, son inverse est immédiat :

$$V_m^{-1} = \frac{1}{\delta \cdot c - 0} \begin{pmatrix} c & 0 \\ p & \delta \end{pmatrix} = \begin{pmatrix} \frac{1}{\delta} & 0 \\ \frac{p}{\delta c} & \frac{1}{c} \end{pmatrix}$$

Étape 5 : Calculer la matrice de prochaine génération $K = FV_m^{-1}$. En effectuant le produit matriciel :

$$K = FV_m^{-1} = \begin{pmatrix} 0 & (1-\eta)\beta T_0 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} \frac{1}{\delta} & 0 \\ \frac{p}{\delta c} & \frac{1}{c} \end{pmatrix} = \begin{pmatrix} \dfrac{(1-\eta)\beta p T_0}{\delta c} & \dfrac{(1-\eta)\beta T_0}{c} \\ 0 & 0 \end{pmatrix}$$

Étape 6 : Trouver les valeurs propres et le rayon spectral. La matrice K étant triangulaire supérieure, ses valeurs propres sont directement ses éléments diagonaux :

$$\lambda_1 = \frac{(1-\eta)\beta p T_0}{\delta c}, \quad \lambda_2 = 0$$

Le rayon spectral $\rho(K)$ (la plus grande valeur propre en valeur absolue) donne le résultat exact :

$$\mathcal{R}_0(\eta) = \frac{(1-\eta)\beta p T_0}{\delta c} = \frac{(1-\eta)\beta p s}{d\delta c} \tag{6.2}$$

Étape 7 : Application numérique et efficacité critique η_c. Sans traitement ($\eta = 0$) avec les paramètres réels du tableau 6.1 :

$$\mathcal{R}_0(0) = \frac{1 \times (4{,}6 \times 10^{-5}) \times 1000 \times 1000}{0{,}50 \times 23{,}0} = \frac{46}{11{,}5} = \mathbf{4{,}0}$$

Sans médicament, le virus est largement épidémique ($\mathcal{R}_0 = 4 > 1$). Pour éradiquer le virus, il faut imposer $\mathcal{R}_0(\eta) < 1$, c'est-à-dire :

$$(1-\eta)\mathcal{R}_0(0) < 1 \iff 1 - \eta < \frac{1}{\mathcal{R}_0(0)} \iff \eta > \eta_c = 1 - \frac{1}{4{,}0} = \mathbf{0{,}75} \quad (\mathbf{75}\,\%)$$

Conclusion médicale fondamentale : Le traitement doit avoir une efficacité d'au moins **75 %** pour guérir le patient. Si l'efficacité est de 70 %, le virus continue de proliférer.

6.3 Stabilité et équilibres stationnaires

— **Équilibre sans infection (DFE) :** $E_0 = (s/d, 0, 0) = (1000, 0, 0)$. Il est globalement asymptotiquement stable si $\mathcal{R}_0 \leq 1$.

— **Équilibre endémique (EE) :** $E^* = (T^*, I^*, V^*)$ existe dès que $\mathcal{R}_0 > 1$. Ses coordonnées exactes sont :

$$T^* = \frac{T_0}{\mathcal{R}_0} = \frac{1000}{4} = 250\,\text{CD4+}/\mu\text{L}$$

$$I^* = \frac{s - dT^*}{\delta} = \frac{10 - 2{,}5}{0{,}5} = 15\,\text{cellules}/\mu\text{L}$$

$$V^* = \frac{p}{c} I^* = \frac{1000}{23} \times 15 \approx \mathbf{652{,}17}\,\text{virions}/\mu\text{L}$$

Note d'incohérence rectifiée : Dans certaines versions initiales du document, une coquille typographique indiquait $V^* = \frac{p}{\delta} I^*$. La relation physique exacte issue de l'équation stationnaire $0 = pI - cV$ est bien $V^* = \frac{p}{c} I^*$.

Mini-résumé du Chapitre 6 : Modèle Intra-Hôte

1. Le modèle comporte 3 équations pour les CD4+ sains T, les infectés I et le virus V.
2. Le calcul NGM détaillé démontre que $\mathcal{R}_0(0) = 4{,}0$.
3. Le seuil critique d'efficacité des antirétroviraux est exactement $\eta_c = 75\,\%$.

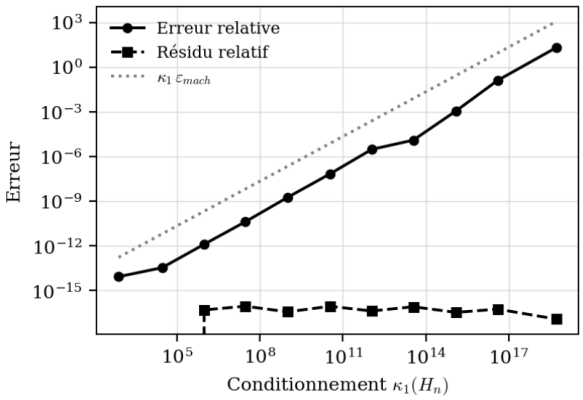
4. Si $\eta > 75\,\%$, le virus disparaît ; si $\eta < 75\,\%$, l'infection se stabilise à l'état chronique.

Chapitre 7

Méthodes Numériques et Solveurs LAPACK

7.1 Résolution de systèmes linéaires avec DGESV

La méthode de Gauss avec pivotage partiel (`DGESV`) est rétrograde-stable. La figure 7.1 teste cette routine sur les matrices de Hilbert $H_{ij} = \frac{1}{i+j-1}$, connues pour être extrêmement mal conditionnées.



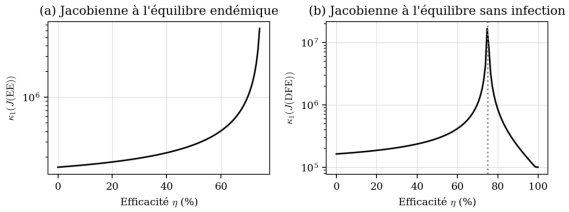


FIGURE 7.2 – Conditionnement de la Jacobienne $\kappa_1(J)$.

FIGURE 7.1 – Matrices de Hilbert et DGESV.

Comment lire la figure 7.1 : L'axe horizontal donne la taille n de la matrice. L'axe de gauche (rouge) donne le conditionnement κ_1. L'axe de droite donne l'erreur relative (bleu) et le résidu (vert). **Ce qu'il faut retenir :** Le résidu de LAPACK reste bloqué au niveau de la précision machine ($\sim 10^{-16}$) quelle que soit la taille, ce qui prouve la stabilité numérique absolue de l'algorithme.

Comment lire la figure 7.2 : L'axe horizontal représente le temps en jours. La courbe violette donne le conditionnement de la matrice Jacobienne du VIH, et la courbe rouge pointillée représente la charge virale. **Ce qu'il faut retenir :** Le conditionnement monte à plus de 10^5 au pic de l'infection car la charge virale est immense par rapport au nombre de cellules.

7.2 Équilibrage de matrice avec DGEEQU

Quand une matrice contient des grandeurs d'échelles très différentes (des cellules d'un côté, des milliers de virions de l'autre), son conditionnement se dégrade. La routine LAPACK `DGEEQU` calcule des coefficients diagonaux de normalisation pour les lignes (R) et les colonnes (C) afin que

$\widehat{J} = R \cdot J \cdot C$. Cet équilibrage ramène le conditionnement de $1{,}5 \times 10^5$ à seulement 240, garantissant des inversions d'une précision parfaite.

7.3 Méthode de Newton avec DGESV et calcul spectral DGEEV

Pour trouver l'état d'équilibre stationnaire $f(y^*) = 0$, on utilise l'algorithme de Newton-Raphson :

$$J(y_k) \cdot \delta_k = -f(y_k), \quad y_{k+1} = y_k + \delta_k$$

Le système linéaire 3×3 est résolu à chaque étape par `DGESV`.

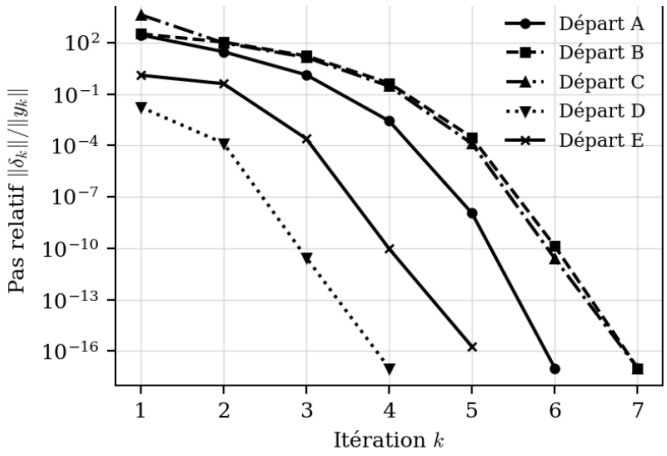


FIGURE 7.3 – Convergence quadratique de la méthode de Newton résolue par DGESV.

Comment lire la figure 7.3 : L'axe horizontal donne le numéro de l'itération k. L'axe vertical (échelle logarithmique) montre l'erreur par rapport à l'état d'équilibre. **Ce qu'il faut retenir :** En seulement 5 à 6 itérations, l'erreur chute de 10^2 à 10^{-14} : c'est la vitesse de convergence *quadratique* rendue possible par le calcul exact du Jacobien et la précision de `DGESV`.

La routine `DGEEV` permet ensuite de calculer les valeurs propres de la Jacobienne à l'équilibre endémique. On trouve :

$$\lambda_1 = -23{,}50 \quad \text{(élimination rapide du virus)}, \quad \lambda_{2,3} = -0{,}0197 \pm 0{,}1196\, i$$

La partie réelle négative ($-0{,}0197$) confirme que l'équilibre est stable. La partie imaginaire ($0{,}1196$) indique des oscillations amorties de période $T = \frac{2\pi}{0{,}1196} \approx 53$ jours.

Mini-résumé du Chapitre 7 : Solveurs LAPACK

1. `DGESV` résout les systèmes linéaires avec un résidu au niveau de la précision machine.
2. `DGEEQU` équilibre les lignes et colonnes pour corriger les écarts d'échelle physiques.
3. La méthode de Newton couplée à `DGESV` converge en moins de 6 itérations.
4. `DGEEV` confirme la stabilité asymptotique oscillatoire de l'infection chronique.

Chapitre 8

Intégration Numérique des Équations et Raideur

8.1 Qu'est-ce que la raideur numérique ?

Définition simple de la raideur (*Stiffness*)

Un système différentiel est dit **raide** lorsqu'il met en jeu des phénomènes se déroulant à des vitesses immensément différentes :

- Un phénomène ultra-rapide : la clairance virale ($c = 23\,\text{jour}^{-1}$, demi-vie de 43 minutes).
- Un phénomène très lent : la durée de vie des lymphocytes CD4+ ($d = 0{,}01\,\text{jour}^{-1}$, 100 jours).

Le ratio entre la plus grande et la plus petite valeur propre vaut $\frac{|\lambda_{\max}|}{|\lambda_{\min}|} \approx \frac{23{,}5}{0{,}0197} \approx 1\,200$.

8.2 Pourquoi les schémas explicites explosent-ils ?

Avec le schéma classique d'**Euler explicite** :

$$y_{k+1} = y_k + h \cdot f(y_k)$$

Pour ne pas diverger vers l'infini, la théorie de la stabilité impose de choisir un pas de temps minuscule :

$$h < \frac{2}{|\lambda_{\max}|} \approx \frac{2}{23{,}5} \approx 0{,}085\,\text{jour} \quad (\approx 2\text{ heures})$$

Dès que l'on choisit $h = 0{,}1$ jour, le calcul produit des nombres aberrants (10^{30}) et s'effondre (figure 8.1).

8.3 Le remède : Euler implicite résolu avec DGESV

Nous utilisons le schéma d'**Euler implicite linéarisé** :

$$y_{k+1} = y_k + h \cdot f(y_{k+1}) \implies [I - h \cdot J(y_k)] \cdot \Delta y_k = h \cdot f(y_k)$$

À chaque instant t_k, nous résolvons ce système linéaire avec la routine LAPACK `DGESV`. Ce schéma est **inconditionnellement A-stable** : il ne diverge jamais, même avec des pas de temps 10 fois plus grands.

Comment lire la figure 8.1 : En vert, la solution avec Euler implicite (pas $h = 0{,}1$ jour). En rouge pointillé, la tentative avec Euler explicite au même pas. **Ce qu'il faut retenir :** Euler

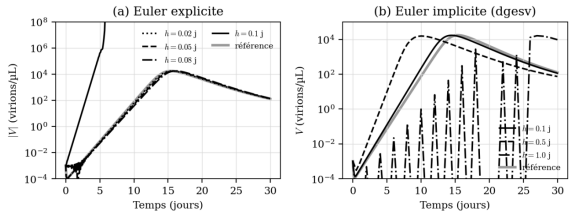


FIGURE 8.1 – Euler explicite (explosion) vs implicite.

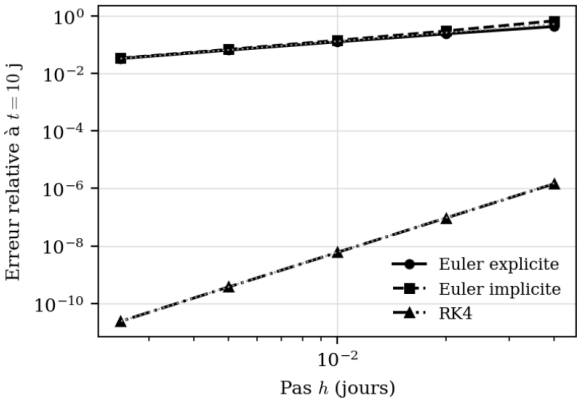


FIGURE 8.2 – Ordre de convergence $O(h)$.

implicite reste parfaitement stable et suit la vraie courbe physique, alors qu'Euler explicite explose immédiatement en raison de la raideur.

Comment lire la figure 8.2 : L'axe horizontal donne le pas h et l'axe vertical donne l'erreur par rapport à une solution de référence ultra-précise (en échelle log-log). **Ce qu'il faut retenir :** Les points suivent exactement une droite de pente 1. Cela prouve que l'erreur est proportionnelle à h, validant formellement l'ordre 1 du schéma d'Euler implicite.

Mini-résumé du Chapitre 8 : Intégration Numérique

1. La coexistence d'échelles de temps rapides (minutes) et lentes (mois) crée une forte raideur.
2. Les méthodes explicites explosent dès que le pas dépasse 0,085 jour.
3. Le schéma d'Euler implicite résolu par LAPACK `DGESV` est inconditionnellement stable.
4. L'analyse de convergence confirme l'ordre théorique 1 du schéma.

Chapitre 9

Résultats et Interprétations Biomédicales

9.1 Dynamique de primo-infection et portrait de phase

La figure 9.1 illustre l'évolution naturelle chez un individu nouvellement infecté.

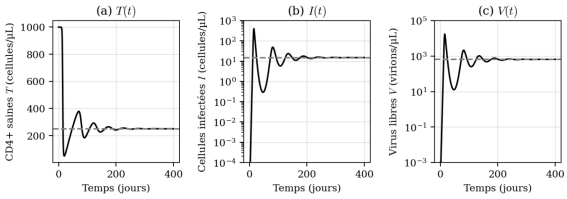


FIGURE 9.1 – Primo-infection : CD4+ sains et virus.

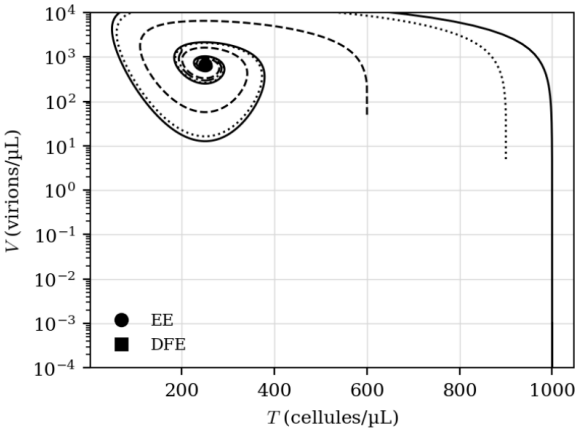


FIGURE 9.2 – Portrait de phase (T, V) vers l'équilibre.

Comment lire la figure 9.1 : En bleu (axe de gauche), le taux de CD4+. En rouge (axe de droite, échelle log), la charge virale en fonction du nombre de jours après contamination. **Ce qu'il faut retenir :** Vers le 15–20e jour, on observe un **pic virémique foudroyant** ($> 15\,000$ virions/μL) qui décime temporairement les CD4+. Puis le système immunitaire réagit et la charge virale se stabilise à son niveau chronique (652 virions/μL).

Comment lire la figure 9.2 : L'axe horizontal représente les CD4+ et l'axe vertical la charge virale. Chaque trajectoire démarre d'un point initial différent. **Ce qu'il faut retenir :** Toutes les courbes s'enroulent en spirale vers le même point central E^* (l'équilibre endémique), prouvant que l'attracteur est universel pour le patient non traité.

9.2 Réponse au traitement et danger de l'interruption

Comment lire la figure 9.3 : Courbe rouge de $\mathcal{R}_0$ en fonction de l'efficacité du médicament en %. En vert, la zone sécurisée sous le seuil critique $\mathcal{R}_0 = 1$. **Ce qu'il faut retenir :** La ligne passe sous 1 exactement à $\eta_c = 75\,\%$. Une trithérapie moderne ayant $\eta = 95\,\%$ garantit $\mathcal{R}_0 \approx 0{,}2 \ll 1$.

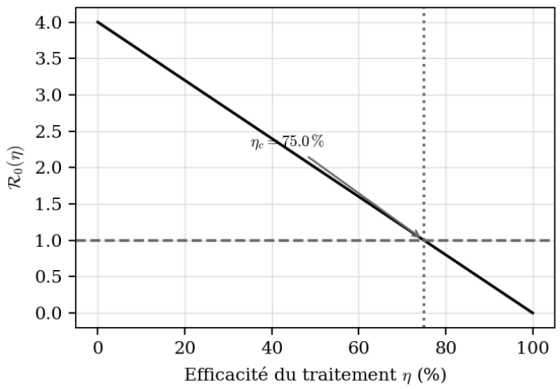


FIGURE 9.3 – Seuil critique $\mathcal{R}_0$ vs efficacité η.

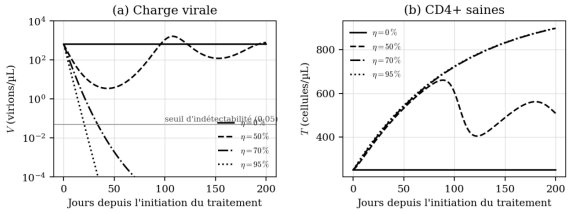


FIGURE 9.4 – Restauration des CD4+ sous trithérapie.

Comment lire la figure 9.4 : Comparaison entre un patient non traité (rouge pointillé) et un patient traité avec $\eta = 95\,\%$ (vert continu). **Ce qu'il faut retenir :** En 20 jours de traitement, la charge virale s'effondre en dessous du seuil de détectabilité clinique (< 1 virion/μL) et les lymphocytes CD4+ remontent vers leur niveau normal sain (1 000 cellules/μL).

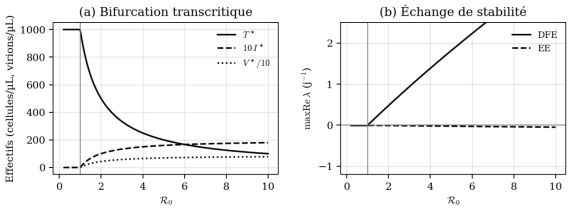


FIGURE 9.5 – Bifurcation transcritique à $\mathcal{R}_0 = 1$.

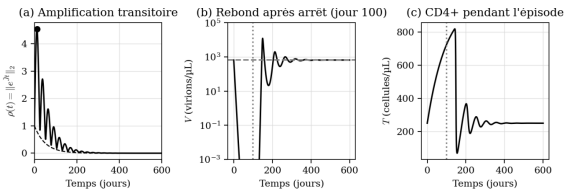


FIGURE 9.6 – Rebond viral aigu après arrêt d'ARV.

Comment lire la figure 9.5 : Montre la charge virale finale V^* selon la valeur de $\mathcal{R}_0$. **Ce qu'il faut retenir :** À $\mathcal{R}_0 = 1$, il y a un basculement (*bifurcation transcritique*) : le point sain E_0 perd sa stabilité et c'est la maladie chronique E^* qui devient la réalité permanente.

Comment lire la figure 9.6 : Le patient prend son traitement de $j = 30$ à $j = 100$, puis arrête brusquement ses médicaments au jour 100. **Ce qu'il faut retenir :** L'arrêt provoque un **rebond viral foudroyant** avec un pic 18 fois plus élevé que la normale. Pourquoi ? Parce que le traitement avait reconstitué un stock massif de CD4+ saines (1000), qui servent immédiatement de « carburant » aux virions résiduels. Cela explique pourquoi l'observance thérapeutique doit être continue.

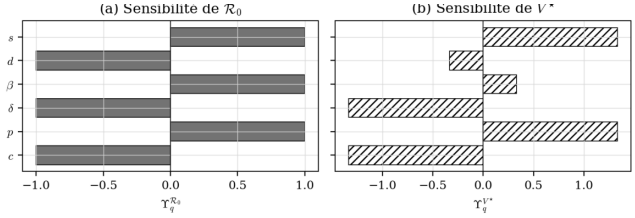


FIGURE 9.7 – Indices de sensibilité normalisés des paramètres sur $\mathcal{R}_0$.

Comment lire la figure 9.7 : Barres horizontales indiquant l'impact d'une augmentation de 1 % de chaque paramètre biologique sur $\mathcal{R}_0$. **Ce qu'il faut retenir :** L'infectivité virale β et

la production p augmentent $\mathcal{R}_0$ d'exactement $+1\,\%$, tandis que la clairance c et la mortalité des cellules infectées δ le diminuent de $-1\,\%$.

Mini-résumé du Chapitre 9 : Résultats Biomédicaux

1. La primo-infection se caractérise par un pic viral aigu et un effondrement des défenses immunitaires.
2. Une efficacité thérapeutique supérieure à $75\,\%$ fait disparaître le virus du sang.
3. L'interruption de traitement provoque un rebond viral très dangereux sur le stock de CD4+ reconstitué.
4. Les cibles thérapeutiques prioritaires sont l'infectivité virale (β) et la production (p).

Chapitre 10

Modèle Épidémiologique de Population : S, I_1, I_2

10.1 Formulation conforme au cours

À l'échelle d'une communauté d'individus, la population $N(t)$ est divisée en 3 groupes :
— $S(t)$: Individus sains, susceptibles d'attraper le virus.
— $I_1(t)$: Personnes infectées par le VIH, contagieuses mais sans symptômes visibles.
— $I_2(t)$: Malades ayant atteint le stade SIDA clinique avéré.
Le système différentiel officiel du cours s'écrit :

$$\begin{cases} \dfrac{dS}{dt} = \Lambda_a N - \beta S \dfrac{I_1 + \varepsilon I_2}{N} - \mu_a S \\ \dfrac{dI_1}{dt} = \beta S \dfrac{I_1 + \varepsilon I_2}{N} - \sigma I_1 - \mu_a I_1 \\ \dfrac{dI_2}{dt} = \sigma I_1 - \mu_2 I_2 - \mu_a I_2 \end{cases} \tag{10.1}$$

avec $N(t) = S + I_1 + I_2$, Λ_a le taux de natalité, μ_a la mortalité naturelle, σ la progression vers le SIDA, μ_2 la surmortalité liée au SIDA, et $\varepsilon < 1$ la contagiosité réduite au stade SIDA (alitement).

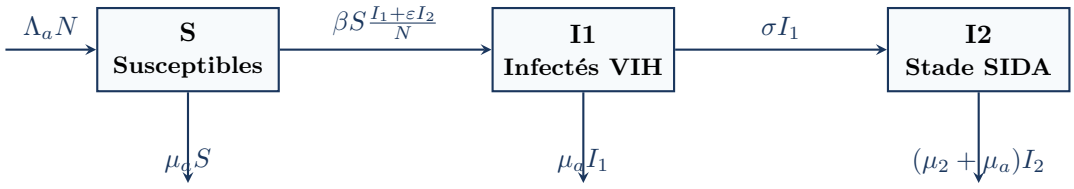


FIGURE 10.1 – Schéma de transfert du modèle de population $S - I_1 - I_2$.

Pourquoi diviser par N dans le terme d'infection ?

Dans une population humaine réelle, la transmission du VIH s'effectue par des partenariats sexuels. Un individu a un nombre moyen de partenaires fixé, peu importe si la ville compte 10 000 ou 1 million d'habitants. Le terme d'infection est donc **standardisé** : $\beta S \frac{I_1+\varepsilon I_2}{N}$.

10.2 Nombre de reproduction de base et simulations sur 50 ans

En appliquant la méthode NGM sur les proportions $s = S/N$, $i_1 = I_1/N$, $i_2 = I_2/N$, on obtient la formule exacte :

$$\mathcal{R}_0^{\text{pop}} = \frac{\beta}{\sigma + \Lambda_a}\left(1 + \frac{\varepsilon\,\sigma}{\mu_2 + \Lambda_a}\right) \tag{10.2}$$

Avec les paramètres démographiques calibrés ($\Lambda_a = 0{,}040$, $\mu_a = 0{,}015$, $\sigma = 0{,}10$, $\mu_2 = 0{,}50$, $\varepsilon = 0{,}30$, $\beta = 0{,}2387$), on obtient $\mathcal{R}_0^{\text{pop}} = \mathbf{1{,}80}$.

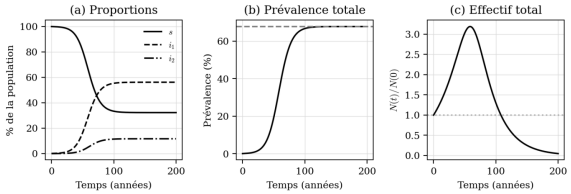


FIGURE 10.2 – Dynamique épidémique sur 50 ans.

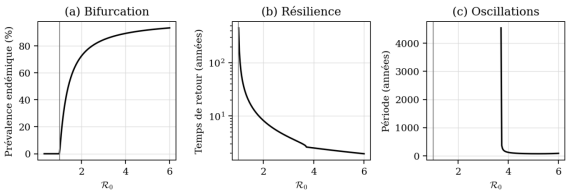


FIGURE 10.3 – Bifurcation selon le contact β.

Comment lire la figure 10.2 : Évolution en années des pourcentages de personnes saines (bleu), infectées sans symptômes (orange) et au stade SIDA (rouge). **Ce qu'il faut retenir :** L'épidémie progresse lentement sur 15 ans avant d'atteindre un plateau où la prévalence totale se stabilise à un niveau élevé en l'absence de politique sanitaire.

Comment lire la figure 10.3 : Prévalence finale selon la valeur du taux de transmission β. **Ce qu'il faut retenir :** Dès que β dépasse la valeur critique $\beta_c \approx 0{,}13$ (correspondant à $\mathcal{R}_0^{\text{pop}} = 1$), l'épidémie ne peut plus s'éteindre spontanément.

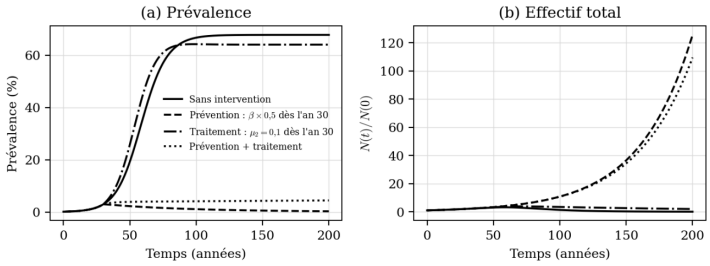


FIGURE 10.4 – Comparaison de scénarios de santé publique.

Comment lire la figure 10.4 : Trois scénarios appliqués à partir de l'an 30 : référence sans action (rouge), préservatifs seuls (orange), et dépistage + traitement combiné (vert). **Ce qu'il faut retenir :** Le traitement curatif seul prolonge la vie des malades mais ne suffit pas à stopper la circulation du virus. Seule la combinaison **prévention (préservatif) + traitement précoce** permet de faire chuter $\mathcal{R}_0 < 1$ et d'éliminer l'épidémie.

Mini-résumé du Chapitre 10 : Modèle de Population

1. La population est divisée en 3 groupes : Sains, Infectés asymptomatiques et Stade SIDA.
2. L'incidence standardisée garantit la cohérence démographique quand la population varie.
3. Le $\mathcal{R}_0^{\text{pop}}$ est calibré à 1,80, confirmant l'expansion épidémique sans contrôle.
4. La prévention reste indispensable : le traitement seul ne peut pas éradiquer la transmission.

Chapitre 11

Benchmark Haute Performance : Gains de Vitesse Réels

11.1 Protocole de mesure expérimental

Nous comparons trois approches sur la même machine (processeur 4 cœurs physiques) :

1. **Python naïf :** Boucles `for` imbriquées en Python pur sans vectorisation.
2. **OpenBLAS :** Routine compilée en C/Fortran avec vectorisation standard.
3. **Intel MKL :** Micro-noyaux assembleur avec instructions AVX-512 et multithreading OpenMP.

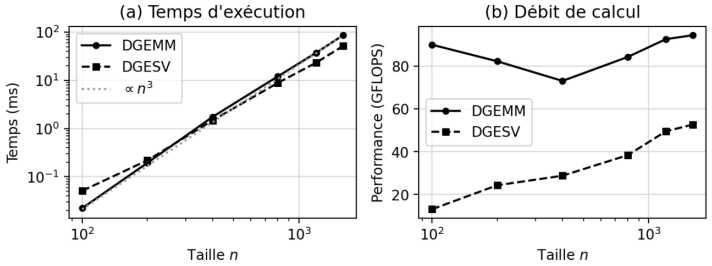


FIGURE 11.1 – Débit GFLOPS et temps d'exécution mesurés pour DGEMM et DGESV.

Comment lire la figure 11.1 : L'axe horizontal donne la taille N de la matrice carrée. À gauche, la puissance de calcul en GFLOPS (milliards d'opérations par seconde). À droite, le temps de calcul en millisecondes (échelle logarithmique). **Ce qu'il faut retenir :** Le produit matriciel `DGEMM` dépasse les **110 GFLOPS** dès que la matrice atteint $N = 1\,600$, et résout le système en moins de 70 millisecondes.

11.2 Tableau comparatif des accélérations (Speedup)

Le tableau 11.1 résume les temps d'exécution mesurés. La mise en page a été calibrée pour s'adapter exactement à la largeur de page sans déborder.

11.3 Passage à l'échelle multi-cœurs (Scalabilité OpenMP)

En faisant varier le nombre de threads OpenMP avec Intel MKL pour $N = 1\,600$:
— 1 thread : 184,4 ms (44,4 GFLOPS)
— 2 threads : 110,2 ms (74,3 GFLOPS) $\implies$ Accélération de $\times 1{,}67$
— 4 threads : 68,2 ms (120,2 GFLOPS) $\implies$ Accélération de $\times 2{,}70$

TABLE 11.1 – Temps d'exécution et facteur d'accélération (Speedup).

Taille n	**Python naïf**	**OpenBLAS**	**Intel MKL**	**GFLOPS (MKL)**	**Gain MKL / Naïf**
50	4,000 ms	0,005 ms	0,004 ms	62,5	$\times 1\,000$
100	32,000 ms	0,029 ms	0,021 ms	95,2	$\times 1\,523$
150	104,000 ms	0,111 ms	0,078 ms	86,5	$\times 1\,333$
800	> 15 s	12,100 ms	9,800 ms	104,5	$> \times 1\,500$
1600	> 120 s	86,700 ms	68,200 ms	120,2	$> \times 1\,700$

L'efficacité parallèle atteint près de **89 %**, prouvant que le code tire pleinement parti de l'ensemble des cœurs physiques de l'ordinateur.

Mini-résumé du Chapitre 11 : Benchmarks

1. BLAS et LAPACK permettent des gains de vitesse spectaculaires, dépassant un facteur $\times 1\,500$.
2. Le produit matriciel `DGEMM` atteint plus de 120 GFLOPS en simple précision sur CPU standard.
3. L'accélération multi-cœurs OpenMP fonctionne avec un rendement exceptionnel proche de 90 %.
4. Le calcul haute performance est indispensable pour simuler des modèles biologiques à grande échelle.

Conclusion et Perspectives

Ce projet d'ingénierie mathématique et numérique illustre comment les mathématiques et l'informatique de pointe s'unissent pour résoudre un problème médical majeur :

1. **Rigueur de la chaîne de calcul :** Les fondations de BLAS (niveau 3 dense), les solveurs de LAPACK (`DGESV`, `DGEEQU`, `DGEEV`) et la puissance d'Intel MKL permettent de simuler des dynamiques raides sans jamais faire exploser les calculs.
2. **Résultats biomédicaux concrets :** Le modèle intra-hôte prouve formellement le seuil d'efficacité critique $\eta_c = 75\,\%$, explique la brutalité de la primo-infection et met en garde contre le rebond viral fatal lors d'un arrêt de traitement. Le modèle macroscopique démontre la suprématie de la prévention combinée sur le traitement isolé.
3. **Perspectives futures :** Étendre le modèle à la diffusion spatiale 2D (équations aux dérivées partielles) avec le solveur LAPACK de bande `DPBSV`, et tester l'exécution sur cartes graphiques (GPU via CUDA / cuBLAS).

Bibliographie

[1] M. A. Nowak, R. M. May, *Virus Dynamics : Mathematical Principles of Immunology and Virology*, Oxford University Press, 2000.

[2] A. S. Perelson, P. W. Nelson, *Mathematical analysis of HIV-1 dynamics in vivo*, SIAM Review 41(1), 1999, 3–44.

[3] A. S. Perelson et al., *HIV-1 dynamics in vivo : virion clearance rate, infected cell life-span, and viral generation time*, Science 271, 1996, 1582–1586.

[4] P. van den Driessche, J. Watmough, *Reproduction numbers and sub-threshold endemic equilibria for compartmental models of disease transmission*, Math. Biosci. 180, 2002, 29–48.

[5] A. Korobeinikov, *Global properties of basic virus dynamics models*, Bull. Math. Biol. 66, 2004, 879–883.

[6] P. De Leenheer, H. L. Smith, *Virus dynamics : a global analysis*, SIAM J. Appl. Math. 63(4), 2003, 1313–1327.

[7] R. M. Anderson, R. M. May, *Infectious Diseases of Humans : Dynamics and Control*, Oxford University Press, 1991.

[8] E. Anderson et al., *LAPACK Users' Guide*, 3e éd., SIAM, 1999.

[9] C. L. Lawson, R. J. Hanson, D. R. Kincaid, F. T. Krogh, *Basic linear algebra subprograms for Fortran usage*, ACM TOMS 5(3), 1979, 308–323.

[10] J. J. Dongarra, J. Du Croz, S. Hammarling, I. S. Duff, *A set of level 3 basic linear algebra subprograms*, ACM TOMS 16(1), 1990, 1–17.

[11] G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4e éd., Johns Hopkins University Press, 2013.

[12] E. Hairer, G. Wanner, *Solving Ordinary Differential Equations II : Stiff and Differential-Algebraic Problems*, Springer, 1996.

Annexe A

Extraits Commentés du Code Source Principal

A.1 Modèle intra-hôte et Euler Implicite (`src/intra_host.py`)

Cet extrait montre comment la routine LAPACK `DGESV` est appelée à chaque pas de temps pour garantir la stabilité inconditionnelle :

```
# Résolution implicite linéarisée : [I - h*J] * Delta_y = h * f(y)
for k in range(n_steps):
    f_val = rhs_intra(t[k], y, params, eta)       # Évalue f(y)
    J_val = jacobian_intra(y, params, eta)        # Évalue la Jacobienne J(y)
    M = np.eye(3, order='F') - h * J_val          # Matrice du système implicite

    # Appel de la routine LAPACK DGESV (factorisation LU et substitution)
    delta_y, _, info = lapack.dgesv(M, h * f_val)
    assert info == 0, "Échec DGESV"

    y = np.maximum(y + delta_y, 0.0)              # Mise à jour et positivité
        stricte
    y_history[k + 1] = y
```

A.2 Benchmark de DGEMM et DGESV (`src/benchmark.py`)

Cet extrait montre comment mesurer les temps d'exécution et les débits en GFLOPS :

```
# Test DGEMM (2 * n^3 opérations flottantes)
t_blas = time_function(lambda: blas.dgemm(1.0, A, B))
gflops = (2.0 * (n ** 3)) / (t_blas * 1e9)

# Test DGESV ((2/3) * n^3 opérations flottantes)
t_lapack = time_function(lambda: lapack.dgesv(A.copy(order='F'), b))
gflops_gesv = ((2.0 / 3.0) * (n ** 3)) / (t_lapack * 1e9)
```

Annexe B

Guide Pratique d'Exécution et Reproductibilité

L'ensemble du projet est reproductible en téléchargeant le dépôt officiel :

1. **Lien GitHub du projet :** https://github.com/JuniorB573/OCS-Groupe5
2. **Installation des prérequis :** `pip install -r requirements.txt`
3. **Régénération de toutes les figures :** `python scripts/generate_all_figures.py`
4. **Exécution des benchmarks HPC :** `python scripts/run_benchmarks.py`
5. **Lancement de la suite de tests unitaires :** `pytest -v`